МИНОБРНАУКИ РОССИИ

Воронежский государственный университет
Факультет прикладной математики, информатики и механики
Кафедра математического обеспечения ЭВМ

Цифровая обработка изображений

Лабораторная работа № 2

Выделение контуров и усредняющая фильтрация
Варианты 1 и 2

Выполнил(а): _____

Группа: _____

Преподаватель: Сырых А. С.

Воронеж, 2026

1. Постановка задачи

Вариант 1. Выделение контуров на изображении операторами Лапласа, Робертса и Собела.

Вариант 2. Удаление шума с изображений в пространственной области усредняющими фильтрами. Для испытаний используется гауссов шум, добавленный в графическом редакторе.

По условию для оценки «удовлетворительно» необходимо выполнить два варианта на выбор. В работе выбраны варианты 1 и 2. Реализация выполнена на C++17; поскольку цвет не является предметом исследования, изображения обрабатываются в градациях серого.

Требуется также дописать заданный класс Image: обеспечить чтение и запись PNG, работу с 1–4 каналами, глубокое копирование и безопасный доступ к пикселям. Методы обработки реализуются отдельно без расширения интерфейса Image.

2. Математическое описание

Изображение представляется функцией яркости f(x,y) со значениями от 0 до 255. Для цветного входа яркость вычисляется по формуле (1); у одно- и двухканального изображения используется готовый канал яркости.

$$f = \mathrm{round}(0.299R + 0.587G + 0.114B). \quad (1)$$

На границах используется повторение ближайшего допустимого пикселя. После расчёта результат ограничивается и округляется по формуле (2). Это ограничение диапазона, а не нормализация по максимальному отклику изображения.

$$Q(t) = \mathrm{round}(\min(255, \max(0,t))). \quad (2)$$

2.1. Выделение контуров

Лаплас оценивает вторую производную яркости. Используется маска [0 1 0; 1 −4 1; 0 1 0]. Для отображения берётся модуль результата:

$$L = f(x-1,y)+f(x+1,y)+f(x,y-1)+f(x,y+1)-4f(x,y);$$

$$g(x,y) = Q(|L|). \quad (3)$$

Робертс вычисляет диагональные разности в окне 2×2. Отсутствие усреднения делает его чувствительным к случайным перепадам:

$$a = f(x,y) - f(x+1,y+1);\ b = f(x+1,y) - f(x,y+1);$$

$$g(x,y) = Q(\sqrt{(a^2+b^2)}). \quad (4)$$

Собел вычисляет две компоненты градиента по маскам 3×3. В каждой маске разность по одной координате сочетается со сглаживанием по другой:

$$Sx = [-1\ 0\ 1;\ -2\ 0\ 2;\ -1\ 0\ 1];$$

$$Sy = [-1\ -2\ -1;\ 0\ 0\ 0;\ 1\ 2\ 1];$$

$$Gx = \Sigma_i \Sigma_j\ Sx[j,i]\ f(x+i,y+j);$$

$$Gy = \Sigma_i \Sigma_j\ Sy[j,i]\ f(x+i,y+j);$$

$$g(x,y) = Q(\sqrt{(Gx^2+Gy^2)}),\ i,j = -1,0,1. \quad (5)$$

Порядок обработки: получить полутоновый вход; для каждого пикселя вычислить отклик выбранного оператора; применить (2); записать результат в новое изображение. Исходный массив не изменяется. Пороговая бинаризация не выполняется.

2.2. Усредняющие фильтры

Для арифметического усреднения все k^2 пикселей окна имеют одинаковый вес. Используются нечётные размеры k=3 и k=5, радиус $r=(k-1)/2$:

$$g(x,y) = Q((1/k^2)\ \Sigma_i \Sigma_j\ f(x+i,y+j)),\ i,j = -r,\ldots,r. \quad (6)$$

Гауссов фильтр сильнее учитывает соседей, расположенных ближе к центру. Веса нормируются, чтобы их сумма была равна единице и постоянная яркость сохранялась. Используется окно 5×5 при σ=1:

$$v(i,j) = \exp(-(i^2+j^2)/(2\sigma^2));\ w(i,j) = v(i,j)/\Sigma_i \Sigma_j\ v(i,j);$$

$$g(x,y) = Q(\Sigma_i \Sigma_j\ w(i,j)\ f(x+i,y+j)). \quad (7)$$

Алгоритм усреднения: подготовить веса; нормировать их; пройти по всем пикселям и просуммировать взвешенные значения окна; округлить и записать результат. Функция applyBoxBlur использует равные веса по (6), а applyGaussianBlur — гауссовы веса по (7) и положительное σ.

Для сравнения результата подавления шума с чистым эталоном используются средний квадрат ошибки и PSNR. Меньшая MSE и больший PSNR означают более близкое совпадение яркостей:

$$MSE = (1/(WH)) \Sigma_x \Sigma_y (g(x,y)-f(x,y))^2;$$

$$PSNR = 10 \log_{10}(255^2/MSE), \text{ дБ}. \quad (8)$$

3. Программная реализация

Класс Image хранит данные в std::unique_ptr<uint8_t[]>. Конструктор копирования и присваивание создают независимый массив. Недопустимый доступ к пикселям игнорируется. Для серого входа getColor возвращает яркость в r, a g и b устанавливает в ноль. setColor сохраняет альфа-канал. Чтение и запись PNG используют библиотеки stb из задания.

Функция или файл	Назначение
toGrayscale	Преобразование RGB в яркость по (1).
applyLaplacian, applyRoberts, applySobel	Операторы Лапласа, Робертса и Собела; формулы (3)–(5).
applyBoxBlur, applyGaussianBlur	Арифметическое и гауссово усреднение; формулы (6), (7).
sample и put	Повторение граничных пикселей; ограничение и округление результата.
Source.cpp	Получение путей, запуск методов и проверка записи результатов.

Алгоритмы находятся в Filters.cpp, а интерфейс Image.h сохранён. Точка входа main находится в Source.cpp. Без аргументов запуска используются test_input/shapes_noisy.png и test_output/shapes_noisy. Два аргумента позволяют выбрать другой входной PNG и каталог результата. Один запуск создаёт семь PNG: полутоновый вход, три контурные карты и три результата усреднения. При ошибке чтения или записи выводится сообщение и возвращается код 1.

В Visual Studio открывается Lab.sln, проект Lab назначается запускаемым, затем используется Ctrl+F5. Для проверки класса и фильтров запускается проект Tests. Исходники разделены на папки include, src и tests; подготовленные изображения лежат в test_input, результаты — в test_output. При сборке объектные файлы попадают в build, исполняемые — в bin. Приложение и тесты собираются отдельно, поскольку имеют разные точки входа main.

Выполнено 18 тестовых случаев и 323 проверки, все успешно. Двенадцать случаев взяты из задания, шесть дополнительно проверяют постоянное поле, ступень яркости, импульс, параметры, присваивание и сохранение прозрачности. У ступени 0→40 проверены ответы Лапласа 40, Робертса 57 и Собела 160. У одиночного импульса 255 среднее 3×3 даёт 28, а гауссов фильтр 3×3 при σ=1 — 52.

4. Тестовые изображения

Подготовлены две сцены 256×192 пикселя. Набор А содержит крупные фигуры, однородные области и участок изменения яркости. Набор Б содержит тонкие штрихи, шахматный фрагмент, окружность и диагональ. Такие изображения позволяют оценить не только сглаживание фона, но и сохранение деталей.

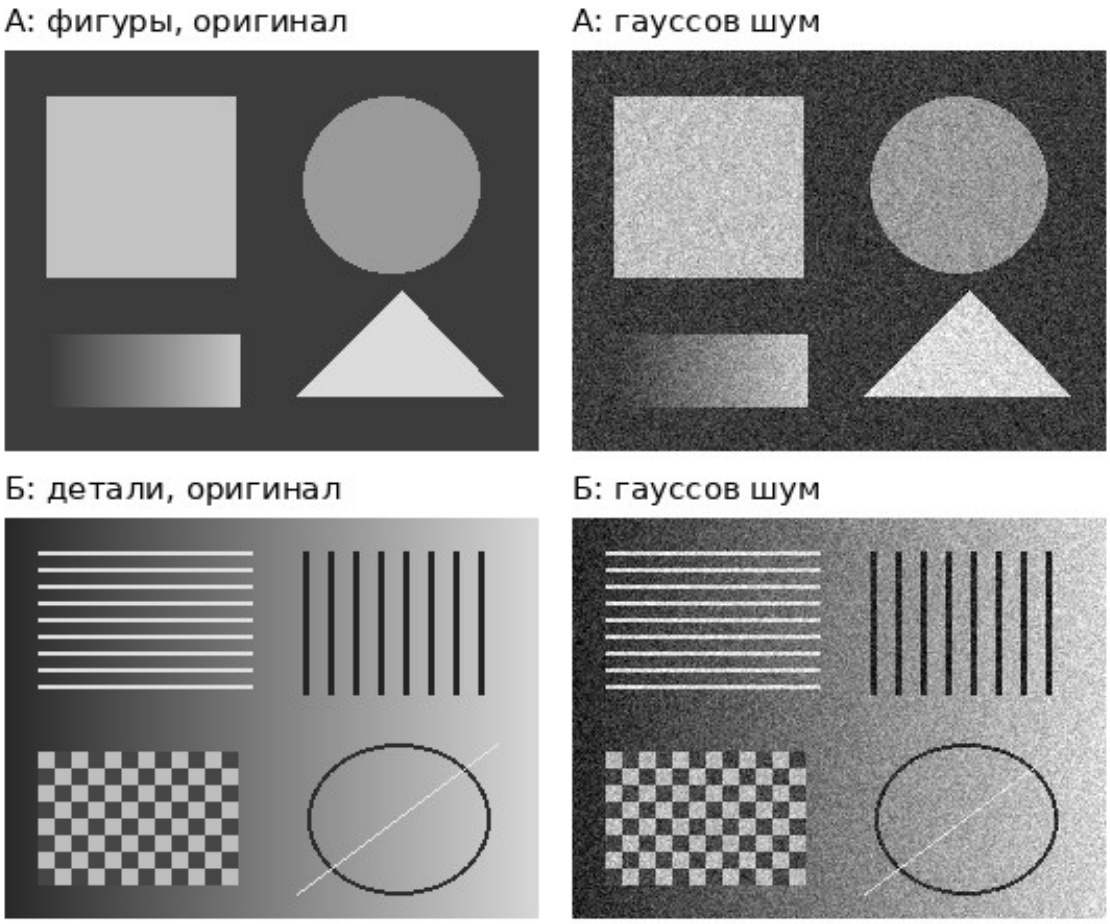


Рисунок 1. Чистые эталоны слева и соответствующие изображения с гауссовым шумом справа.

Гауссов шум добавлен редактором ImageMagick вне C++-программы фильтрации: +noise Gaussian, -attenuate 0.9, глубина 8 бит, начальные значения генератора 731 и 1201. Это соответствует разрешению использовать другой графический редактор в варианте 2. Чистые эталоны сохранены отдельно.

Измеренное стандартное отклонение разности с эталоном составляет 17.92 для набора А и 17.92 для Б. Параметр шума в уровнях яркости отличается от σ пространственного гауссова фильтра.

5. Результаты варианта 1

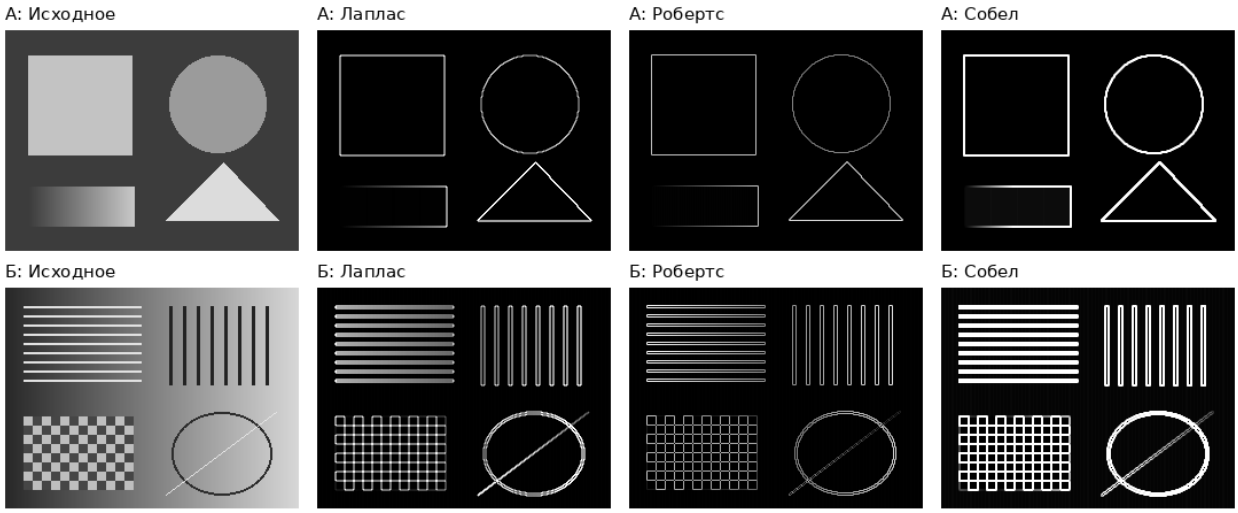


Рисунок 2. Сверху набор А, снизу набор Б. В каждом ряду: вход, Лаплас, Робертс, Собел.

На крупных фигурах все операторы выявляют границы. Лаплас показывает соседние отклики по обе стороны перепада, поскольку модуль второй разности отображает оба её знака. Робертс формирует компактный отклик, Собел — более широкий и яркий. Отличия связаны с размером окна и коэффициентами масок, поэтому одна яркость не определяет качество метода.

На наборе Б видны близкие контуры тонких штрихов и клеток. На линейном градиенте внутренняя вторая разность близка к нулю, а первые разности реагируют на изменение яркости. Округление яркостей может создавать слабый остаточный отклик.

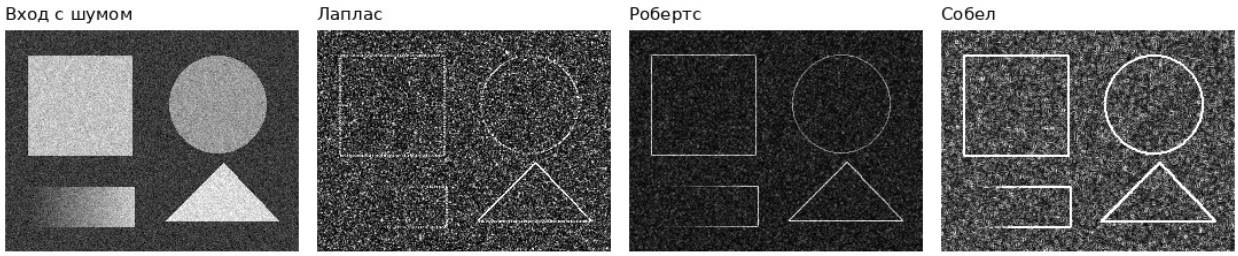


Рисунок 3. Набор А с гауссовым шумом и результаты Лапласа, Робертса и Собела.

После добавления шума контурные операторы реагируют на случайные перепады внутри однородных областей. У Собела присутствует поперечное

сглаживание, но оно не устраняет шум полностью. Сильные отклики ограничиваются 255, поэтому после насыщения их яркость уже не пропорциональна точной величине производной.

6. Результаты варианта 2

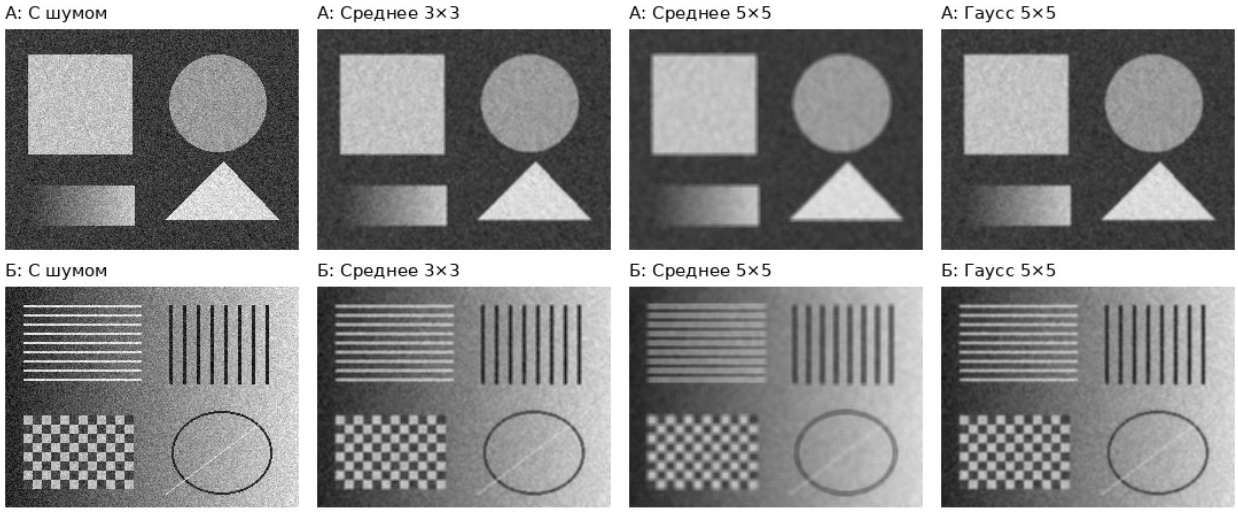


Рисунок 4. Сверху набор А, снизу набор Б. Шумный вход, среднее 3×3, среднее 5×5, Гаусс 5×5 с σ=1.

Метод	А: MSE	А: PSNR, дБ	Б: MSE	Б: PSNR, дБ
Вход с шумом	321.50	23.06	321.63	23.06
Среднее 3×3	111.66	27.65	437.16	21.72
Среднее 5×5	147.53	26.44	873.40	18.72
Гаусс 5×5	95.38	28.34	400.27	22.11

На наборе А все три фильтра уменьшают ошибку по сравнению с шумным входом. Гауссов фильтр даёт наименьшую MSE из рассмотренных усредняющих методов: 95.38. Его удалённые соседи имеют меньшие веса, поэтому границы смешиваются слабее, чем при равномерном окне 5×5.

Увеличение арифметического окна с 3×3 до 5×5 снижает зернистость фона, но повышает MSE на А с 111.66 до 147.53: размытие границ перевешивает дополнительное подавление шума. На наборе Б тонкие элементы сравнимы по ширине с окном. Все три фильтра ухудшают общую MSE относительно шумного входа, поскольку теряется часть полезных деталей.

Таким образом, видимое сглаживание и близость к эталону — разные критерии. Размер окна следует выбирать по содержанию изображения. Из трёх проверенных усредняющих настроек гауссов фильтр 5×5 с σ=1 имеет меньшую ошибку на обеих сценах; это не утверждение о его превосходстве при любых входах.

7. Вывод

Выполнены два выбранных варианта: выделение контуров и подавление гауссова шума усредняющими фильтрами. Реализован заданный класс Image без добавления методов. Приложение обрабатывает четыре демонстрационных входа и сохраняет 28 выходных PNG; все 18 тестовых случаев пройдены.

Лаплас, Робертс и Собел обнаруживают перепады яркости, но также реагируют на шум. Усреднение уменьшает колебания на ровных областях и размывает границы. Увеличение окна не гарантирует улучшения общей ошибки. Полученные результаты согласуются с формулами операторов и подтверждаются сравнением с чистыми эталонами.

Источники

1. Материалы задания: lab_1.pdf, image_env.pdf, ReadMe.pdf, интерфейс Image и тесты tests_image.cpp.

2. Библиотеки stb_image и stb_image_write для чтения и записи PNG из материалов задания.

3. Библиотека doctest и предоставленное тестовое окружение.

4. Справочная документация ImageMagick по добавлению гауссова шума.

Приложение А. Алгоритмы обработки

```
#include "Filters.h"
#include <algorithm>
#include <cmath>
#include <vector>

namespace {
enum class Edge { Laplace, Roberts, Sobel };
void requireGray(const Image& image) {
    if (image.width() <= 0 || image.height() <= 0 || image.nChannels()
!= 1)
        throw std::invalid_argument("Expected a nonempty grayscale
image");
}
void requireWindow(int window) {
    if (window < 1 || window > 31 || window % 2 == 0)
        throw std::invalid_argument("Window must be odd and between 1
and 31");
}
int sample(const Image& image, int x, int y) {
    // Replicate the closest border pixel; do not add artificial black
edges.
    x = std::clamp(x, 0, image.width() - 1);
    y = std::clamp(y, 0, image.height() - 1);
    int r = 0, g = 0, b = 0;
    image.getColor(x, y, r, g, b);
    return r;
}
void put(Image& image, int x, int y, double value) {
    const int v = static_cast<int>(std::lround(std::clamp(value, 0.0,
255.0)));
    image.setColor(x, y, v, v, v);
}
}

namespace Filters {
Image toGrayscale(const Image& input) {
    if (input.width() <= 0 || input.height() <= 0)
        throw std::invalid_argument("Empty input image");
    Image result(input.width(), input.height(), 1);
    for (int y = 0; y < input.height(); ++y)
        for (int x = 0; x < input.width(); ++x) {
            int r = 0, g = 0, b = 0;
            input.getColor(x, y, r, g, b);
            put(result, x, y, input.nChannels() < 3 ? r : 0.299*r +
0.587*g + 0.114*b);
        }
    return result;
}

static Image edges(const Image& gray, Edge method) {
```

```
                                                    requireGray(gray);
                Image    result(gray.width(),    gray.height(),    1);
          const  int  sx[3][3]  =  {{-1,0,1},  {-2,0,2},  {-1,0,1}};
          const  int  sy[3][3]  =  {{-1,-2,-1},  {0,0,0},  {1,2,1}};
             for   (int   y   =   0;   y   <   gray.height();   ++y)
                  for  (int  x  =  0;  x  <  gray.width();  ++x)  {
                                             double    value    =    0;
                                   if   (method   ==   Edge::Laplace)   {
                                 value  =  std::abs(sample(gray,x-1,y)  +
sample(gray,x+1,y)                                                      +
                                                  sample(gray,x,y-1)  +
sample(gray,x,y+1)                 -                4*sample(gray,x,y));
                            }  else  if  (method  ==  Edge::Roberts)  {
                const int a = sample(gray,x,y) - sample(gray,x+1,y+1);
                const int b = sample(gray,x+1,y) - sample(gray,x,y+1);
                                          value   =   std::hypot(a,   b);
                                                       }    else    {
                                             int   gx   =   0,   gy   =   0;
                                        for  (int  j  =  -1;  j  <=  1;  ++j)
                                        for  (int  i  =  -1;  i  <=  1;  ++i)  {
                                         const  int  p  =  sample(gray,x+i,y+j);
                                                   gx  +=  p  *  sx[j+1][i+1];
                                                   gy  +=  p  *  sy[j+1][i+1];
                                                                            }
                                              value   =   std::hypot(gx,   gy);
                                                                            }
                                             put(result,   x,   y,   value);
                                                                            }
                                                    return           result;
}

static  Image  average(const  Image&  gray,  int  window,  double  sigma)  {
                                                    requireGray(gray);
                                                 requireWindow(window);
                  if    (!std::isfinite(sigma)    ||    sigma    <    0)
               throw  std::invalid_argument("Sigma  must  be  finite  and
nonnegative");
                     const     int     radius     =     window     /     2;
              std::vector<double>   weights(window   *   window,   1.0);
                                  double        total        =        0;
               for   (int   j   =   -radius;   j   <=   radius;   ++j)
                    for  (int  i  =  -radius;  i  <=  radius;  ++i)  {
                      double&  w  =  weights[(j+radius)*window+i+radius];
               if (sigma > 0) w = std::exp(-(i*i+j*j)/(2*sigma*sigma));
                                                    total    +=    w;
                                                                            }
                for    (double&    w    :    weights)    w    /=    total;
                  Image    result(gray.width(),    gray.height(),    1);
               for   (int   y   =   0;   y   <   gray.height();   ++y)
                    for  (int  x  =  0;  x  <  gray.width();  ++x)  {
                                                double    sum    =    0;
```

```
                        for (int j = -radius; j <= radius; ++j)
                           for (int i = -radius; i <= radius; ++i)
                           sum += weights[(j+radius)*window+i+radius] *
sample(gray,x+i,y+j);
                                                        put(result,x,y,sum);
                                                                    }
                                                  return            result;
}


Image        applyLaplacian(const        Image&        source)       {
               return    edges(toGrayscale(source),    Edge::Laplace);
}

Image         applyRoberts(const         Image&         source)        {
               return    edges(toGrayscale(source),    Edge::Roberts);
}

Image         applySobel(const         Image&         source)         {
                 return    edges(toGrayscale(source),    Edge::Sobel);
}

Image   applyBoxBlur(const   Image&   source,   int   kernelSize)   {
            return   average(toGrayscale(source),   kernelSize,   0.0);
}

Image applyGaussianBlur(const Image& source, int kernelSize, double
sigma)                                                                {
               if    (!std::isfinite(sigma)    ||    sigma    <=    0)
        throw std::invalid_argument("Gaussian sigma must be finite and
positive");
          return   average(toGrayscale(source),   kernelSize,   sigma);
}

}
```

Приложение Б. Класс Image

```
#define                                          STB_IMAGE_IMPLEMENTATION
#define                                    STB_IMAGE_WRITE_IMPLEMENTATION
#include                                                    "stb_image.h"
#include                                                "stb_image_write.h"
#include                                                         "Image.h"
#include                                                         <cstring>
#include                                                          <limits>

Image::Image()                                =                    default;

Image::Image(int    width,    int    height,    int    nChannels)    {
    if (width <= 0 || height <= 0 || nChannels < 1 || nChannels > 4)
            throw std::invalid_argument("Invalid image dimensions or
channel                                                          count");
         const  size_t  limit  =  std::numeric_limits<size_t>::max();
     if (static_cast<size_t>(width) > limit / height / nChannels ||
              width  >  std::numeric_limits<int>::max()  /  nChannels)
                  throw  std::length_error("Image  is  too  large");
                                      m_width         =         width;
                                    m_height         =         height;
                                 m_nChannels        =        nChannels;
       m_size  =  static_cast<size_t>(width)  *  height  *  nChannels;
                  m_data    =    std::make_unique<uint8_t[]>(m_size);
             if   (nChannels   ==   2   ||   nChannels   ==   4)
         for (size_t i = nChannels - 1; i < m_size; i += nChannels)
                                                 m_data[i]    =    255;
}

Image::Image(const          std::string&          filepath)          {
    if (!isRead(filepath)) throw std::runtime_error("Cannot read: " +
filepath);
}

Image::Image(const                      Image&                     other)
              :    m_width(other.m_width),    m_height(other.m_height),
             m_nChannels(other.m_nChannels),  m_size(other.m_size)  {
                                      if          (m_size)          {
                    m_data   =   std::make_unique<uint8_t[]>(m_size);
              std::memcpy(m_data.get(),  other.m_data.get(),  m_size);
                                                                    }
}

Image&       Image::operator=(const       Image&       other)       {
                   if    (this    ==    &other)    return    *this;
       //  Allocate  before  changing  the  object:  allocation  failure
preserves                              its                          state.
                        auto      copy      =      other.m_size      ?
std::make_unique<uint8_t[]>(other.m_size)          :          nullptr;
            if   (copy)   std::memcpy(copy.get(),   other.m_data.get(),
other.m_size);
```

```
                                m_data        =        std::move(copy);
                                 m_width        =        other.m_width;
                                m_height        =        other.m_height;
                              m_nChannels       =       other.m_nChannels;
                                     m_size         =         other.m_size;
                                                  return            *this;
}

bool       Image::isRead(const       std::string&       filepath)       {
               int    w    =    0,    h    =    0,    channels    =    0;
           std::unique_ptr<stbi_uc,   decltype(&stbi_image_free)>   raw(
                 stbi_load(filepath.c_str(),  &w,  &h,  &channels,  0),
stbi_image_free);
                               if       (!raw)       return       false;
                           Image      loaded(w,      h,      channels);
         std::memcpy(loaded.m_data.get(),  raw.get(),  loaded.m_size);
                                        *this          =          loaded;
                                                  return            true;
}

bool    Image::isWrite(const    std::string&    filepath)    const    {
                           if      (!m_data)       return       false;
          return  stbi_write_png(filepath.c_str(),  m_width,  m_height,
m_nChannels,
                            m_data.get(), m_width * m_nChannels) != 0;
}

void  Image::setColor(int  x,  int  y,  int  r,  int  g,  int  b)  {
     if (!m_data || x < 0 || y < 0 || x >= m_width || y >= m_height)
return;
       const  size_t  i  =  (static_cast<size_t>(y)  *  m_width  +  x)  *
m_nChannels;
                         m_data[i]      =      static_cast<uint8_t>(r);
                           if      (m_nChannels      >=      3)      {
                        m_data[i   +   1]   =   static_cast<uint8_t>(g);
                        m_data[i   +   2]   =   static_cast<uint8_t>(b);
                                                                      }
       // Alpha is not an argument of setColor and is therefore
preserved.
}

void Image::getColor(int x, int y, int& r, int& g, int& b) const {
     if (!m_data || x < 0 || y < 0 || x >= m_width || y >= m_height)
return;
       const  size_t  i  =  (static_cast<size_t>(y)  *  m_width  +  x)  *
m_nChannels;
                                       r          =          m_data[i];
      // For gray/gray-alpha the intensity is returned in r (as in
supplied                                                        tests).
           g   =   m_nChannels   >=   3   ?   m_data[i   +   1]   :   0;
           b   =   m_nChannels   >=   3   ?   m_data[i   +   2]   :   0;
```

```
}
```